

Unit Testing: Example and Explanation

As you mentioned:

- **Purpose:** Verify individual components (methods, classes) in isolation.
- **Tools:** JUnit 5, Mockito.
- **Process:** Write tests for business logic, utility methods, and service layers. Mock external dependencies (databases, other services) to ensure tests are fast and isolated.

Let's illustrate this with a simple Spring Boot example.

Scenario:

Imagine you have a ProductService that manages product information. This service interacts with a ProductRepository (which would typically talk to a database). For unit testing, we want to test only the ProductService's logic, without actually hitting a real database. This is where Mockito comes in.

1. The Product Model (POJO):

```
// src/main/java/com/example/demo/model/Product.java
package com.example.demo.model;
```

```
import java.util.Objects;
```

```
public class Product {
    private String id;
    private String name;
    private double price;

    public Product(String id, String name, double price) {
        this.id = id;
        this.name = name;
        this.price = price;
    }
}
```

```
// Getters
```

```
public String getId() { return id; }
public String getName() { return name; }
public double getPrice() { return price; }
```

```
// Setters
```

```
public void setId(String id) { this.id = id; }
```

```

public void setName(String name) { this.name = name; }
public void setPrice(double price) { this.price = price; }

// Override equals and hashCode for proper object comparison in tests
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (o == null || getClass() != o.getClass()) return false;
    Product product = (Product) o;
    return Double.compare(product.price, price) == 0 &&
        Objects.equals(id, product.id) &&
        Objects.equals(name, product.name);
}

@Override
public int hashCode() {
    return Objects.hash(id, name, price);
}

@Override
public String toString() {
    return "Product{" +
        "id=" + id + "\" +
        ", name=" + name + "\" +
        ", price=" + price +
        "}";
}
}

```

2. The ProductRepository Interface (Simulating Database Access):

```

// src/main/java/com/example/demo/repository/ProductRepository.java
package com.example.demo.repository;

import com.example.demo.model.Product;
import java.util.Optional;
import java.util.List;

```

```
// In a real Spring Boot app, this would extend JpaRepository or similar
public interface ProductRepository {
    Optional<Product> findById(String id);
    List<Product> findAll();
    Product save(Product product);
    void deleteById(String id);
}
```

3. The ProductService (Business Logic to be Tested):

```
// src/main/java/com/example/demo/service/ProductService.java
package com.example.demo.service;
```

```
import com.example.demo.model.Product;
import com.example.demo.repository.ProductRepository;
import org.springframework.stereotype.Service;
```

```
import java.util.List;
import java.util.Optional;
```

```
@Service
public class ProductService {
```

```
    private final ProductRepository productRepository;
```

```
    // Spring will inject ProductRepository here
```

```
    public ProductService(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }
```

```
    public List<Product> getAllProducts() {
        return productRepository.findAll();
    }
```

```
    public Optional<Product> getProductById(String id) {
        return productRepository.findById(id);
    }
```

```

public Product createProduct(Product product) {
    // Basic validation example
    if (product.getPrice() <= 0) {
        throw new IllegalArgumentException("Product price must be positive.");
    }
    return productRepository.save(product);
}

public Optional<Product> updateProduct(String id, Product updatedProduct) {
    return productRepository.findById(id).map(existingProduct -> {
        existingProduct.setName(updatedProduct.getName());
        existingProduct.setPrice(updatedProduct.getPrice());
        return productRepository.save(existingProduct);
    });
}

public boolean deleteProduct(String id) {
    if (productRepository.findById(id).isPresent()) {
        productRepository.deleteById(id);
        return true;
    }
    return false;
}
}

```

4. The Unit Test for ProductService:

```

// src/test/java/com/example/demo/service/ProductServiceTest.java
package com.example.demo.service;

import com.example.demo.model.Product;
import com.example.demo.repository.ProductRepository;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;

```

```

import org.mockito.junit.jupiter.MockitoExtension;

import java.util.Arrays;
import java.util.List;
import java.util.Optional;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class) // Enables Mockito annotations for JUnit 5
@DisplayName("ProductService Unit Tests")
public class ProductServiceTest {

    @Mock // Creates a mock instance of ProductRepository
    private ProductRepository productRepository;

    @InjectMocks // Injects the mocks into ProductService
    private ProductService productService;

    private Product product1;
    private Product product2;

    @BeforeEach // This method runs before each test method
    void setUp() {
        // Initialize common test data
        product1 = new Product("P001", "Laptop", 1200.00);
        product2 = new Product("P002", "Mouse", 25.00);
    }

    @Test
    @DisplayName("Should retrieve all products")
    void getAllProducts_shouldReturnAllProducts() {
        // Given: Define the behavior of the mocked repository
        when(productRepository.findAll()).thenReturn(Arrays.asList(product1, product2));

        // When: Call the method under test
        List<Product> products = productService.getAllProducts();

        // Then: Assert the results
    }
}

```

```

    assertNotNull(products);
    assertEquals(2, products.size());
    assertTrue(products.contains(product1));
    assertTrue(products.contains(product2));

    // Verify that the mocked method was called exactly once
    verify(productRepository, times(1)).findAll();
}

@Test
@DisplayName("Should retrieve product by ID when found")
void getProductById_shouldReturnProduct_whenFound() {
    // Given
    when(productRepository.findById("P001")).thenReturn(Optional.of(product1));

    // When
    Optional<Product> foundProduct = productService.getProductById("P001");

    // Then
    assertTrue(foundProduct.isPresent());
    assertEquals(product1, foundProduct.get());
    verify(productRepository, times(1)).findById("P001");
}

@Test
@DisplayName("Should return empty optional when product by ID not found")
void getProductById_shouldReturnEmptyOptional_whenNotFound() {
    // Given
    when(productRepository.findById("P999")).thenReturn(Optional.empty());

    // When
    Optional<Product> foundProduct = productService.getProductById("P999");

    // Then
    assertFalse(foundProduct.isPresent());
    verify(productRepository, times(1)).findById("P999");
}

@Test

```

```

@DisplayName("Should create a new product successfully")
void createProduct_shouldReturnSavedProduct() {
    // Given
    Product newProduct = new Product(null, "Keyboard", 75.00); // ID might be
generated by DB
    Product savedProduct = new Product("P003", "Keyboard", 75.00);
    when(productRepository.save(any(Product.class))).thenReturn(savedProduct);

    // When
    Product result = productService.createProduct(newProduct);

    // Then
    assertNotNull(result.getId());
    assertEquals("Keyboard", result.getName());
    assertEquals(75.00, result.getPrice());
    verify(productRepository, times(1)).save(newProduct); // Verify it was called with
the correct product
}

```

```

@Test
@DisplayName("Should throw IllegalArgumentException for invalid product price")
void createProduct_shouldThrowException_forInvalidPrice() {
    // Given
    Product invalidProduct = new Product(null, "Broken Item", 0.00);

    // When & Then
    // Assert that calling createProduct with invalidProduct throws
IllegalArgumentException
    assertThrows(IllegalArgumentException.class, () ->
productService.createProduct(invalidProduct));

    // Verify that save was never called because validation failed
    verify(productRepository, never()).save(any(Product.class));
}

```

```

@Test
@DisplayName("Should update an existing product")
void updateProduct_shouldReturnUpdatedProduct_whenFound() {
    // Given

```

```
Product updatedDetails = new Product("P001", "Updated Laptop", 1300.00);
when(productRepository.findById("P001")).thenReturn(Optional.of(product1));
when(productRepository.save(any(Product.class))).thenReturn(updatedDetails);
// Mock save to return updated product
```

```
// When
Optional<Product> result = productService.updateProduct("P001",
updatedDetails);
```

```
// Then
assertTrue(result.isPresent());
assertEquals("Updated Laptop", result.get().getName());
assertEquals(1300.00, result.get().getPrice());
verify(productRepository, times(1)).findById("P001");
verify(productRepository, times(1)).save(any(Product.class));
}
```

```
@Test
@DisplayName("Should return empty optional when updating non-existent
product")
void updateProduct_shouldReturnEmptyOptional_whenNotFound() {
    // Given
    Product updatedDetails = new Product("P999", "Non Existent", 100.00);
    when(productRepository.findById("P999")).thenReturn(Optional.empty());
```

```
// When
Optional<Product> result = productService.updateProduct("P999",
updatedDetails);
```

```
// Then
assertFalse(result.isPresent());
verify(productRepository, times(1)).findById("P999");
verify(productRepository, never()).save(any(Product.class)); // Save should not be
called
}
```

```
@Test
@DisplayName("Should delete product by ID when found")
void deleteProduct_shouldReturnTrue_whenFound() {
```



```

// Given
when(productRepository.findById("P001")).thenReturn(Optional.of(product1));
doNothing().when(productRepository).deleteById("P001"); // Mock void method

// When
boolean deleted = productService.deleteProduct("P001");

// Then
assertTrue(deleted);
verify(productRepository, times(1)).findById("P001");
verify(productRepository, times(1)).deleteById("P001");
}

@Test
@DisplayName("Should return false when deleting non-existent product")
void deleteProduct_shouldReturnFalse_whenNotFound() {
    // Given
    when(productRepository.findById("P999")).thenReturn(Optional.empty());

    // When
    boolean deleted = productService.deleteProduct("P999");

    // Then
    assertFalse(deleted);
    verify(productRepository, times(1)).findById("P999");
    verify(productRepository, never()).deleteById(anyString()); // Delete should not be
called
}
}

```

Explanation of the Unit Test:

1. **@ExtendWith(MockitoExtension.class):** This JUnit 5 annotation integrates Mockito with your tests, allowing you to use Mockito annotations like @Mock and @InjectMocks.
2. **@Mock private ProductRepository productRepository;:**
 - This tells Mockito to create a "mock" object for ProductRepository. A mock is a stand-in object that mimics the behavior of a real object.
 - Instead of connecting to a real database, this mock will simulate the

ProductRepository's behavior as we define it.

3. **@InjectMocks private ProductService productService;**
 - This tells Mockito to create an instance of ProductService and inject any @Mock objects (like productRepository) into its constructor or fields.
 - This allows us to test ProductService in isolation, as its dependency (ProductRepository) is controlled by our mock.
4. **@BeforeEach void setUp():**
 - This method runs before *each* test method. It's used to set up common test data or reset the state of mocks to ensure tests are independent.
5. **@Test and @DisplayName:**
 - @Test marks a method as a test case.
 - @DisplayName provides a more readable name for the test in test reports.
6. **when(...).thenReturn(...) (Stubbing):**
 - This is the core of Mockito. It's called "stubbing."
 - when(productRepository.findAll()) specifies that when the findAll() method is called on the productRepository mock...
 - thenReturn(Arrays.asList(product1, product2)) defines what the mock should return (a list of our predefined products).
 - This allows us to control the "external" behavior that ProductService depends on without needing a real database.
7. **doNothing().when(...):**
 - Used for mocking void methods.
doNothing().when(productRepository).deleteById("P001") means that when deleteById is called with "P001", do nothing (i.e., don't throw an error, just proceed).
8. **assertNotNull, assertEquals, assertTrue, assertFalse, assertThrows (Assertions):**
 - These are JUnit 5 assertion methods used to verify the outcome of the method under test.
 - assertEquals(expected, actual) checks if two values are equal.
 - assertTrue(condition) checks if a condition is true.
 - assertThrows(Exception.class, () -> methodCall()) checks if a specific exception is thrown.
9. **verify(mock, times(n)).methodCall() (Verification):**
 - After calling the method under test, verify() is used to ensure that the mocked dependencies were interacted with as expected.
 - verify(productRepository, times(1)).findAll() checks that the findAll() method on the productRepository mock was called exactly once.
 - verify(productRepository, never()).save(any(Product.class)) checks that the

save() method was *never* called.

Why Unit Testing is Important:

- **Early Bug Detection:** Catches bugs in individual components early in the development cycle, before they become integrated into larger systems.
- **Faster Feedback Loop:** Unit tests run quickly, providing immediate feedback on code changes.
- **Isolation:** Tests components in isolation, making it easier to pinpoint the exact source of a bug.
- **Refactoring Confidence:** Gives you confidence to refactor code, knowing that if you break existing functionality, the tests will catch it.
- **Documentation:** Tests serve as living documentation for how individual units of code are supposed to behave.

This example demonstrates how unit tests, using JUnit 5 and Mockito, allow you to thoroughly test your business logic in isolation by mocking external dependencies, ensuring your core application logic is robust and correct.